

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Artificial Intelligence and Data Science****ASSESSMENT TOOL 3 – DESIGN TASK**

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	B.VANI	Reg. No.:	192472348
Section / Batch:	CSE-AI/2024	Date:	

Code of Conduct

I _____B.VANI_(192472348)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____B.VANI_____

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3

Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4
---	--	----

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select $N = 1, 2$, or 3 , display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANSWERS – ASSESSMENT TOOL 3 DESIGN TASK

Natural Language Processing (DSA03) | CO3 – Precision

Answers are added at the end of the supplied assessment document, following the four tasks specified in Annexure A.

Q1. N-gram Based Text Prediction

Objective: Build unigram, bigram, and trigram language models from an English corpus, calculate maximum-likelihood probabilities, predict the top-5 next words, and demonstrate the zero-probability problem of an unsmoothed N-gram model.

Algorithm / Design

1. Preprocess the corpus by converting text to lowercase, splitting it into sentences, and tokenizing words.
2. Count unigram, bigram, and trigram frequencies.
3. Estimate maximum-likelihood probabilities: $P(w) = \text{count}(w)/N$; $P(w_2|w_1) = \text{count}(w_1, w_2)/\text{count}(w_1)$; $P(w_3|w_1, w_2) = \text{count}(w_1, w_2, w_3)/\text{count}(w_1, w_2)$.
4. For an incomplete sentence, rank candidate next words by the selected N-gram probability.
5. An unseen N-gram has probability zero in the unsmoothed model.

Python Implementation

```
import re
from collections import Counter

corpus = [
    "Artificial intelligence is changing the world.",
    "Artificial intelligence is useful in healthcare.",
    "Artificial intelligence is used in education.",
    "Machine learning is a part of artificial intelligence.",
    "Machine learning can analyze data.",
    "Deep learning is useful for language processing."
]

def tokenize(text):
    return re.findall(r"[a-z]+", text.lower())

sentences = [tokenize(s) for s in corpus]
unigram, bigram, trigram = Counter(), Counter(), Counter()

for tokens in sentences:
    unigram.update(tokens)
    bigram.update(zip(tokens, tokens[1:]))
    trigram.update(zip(tokens, tokens[1:], tokens[2:]))

TOTAL = sum(unigram.values())

def P1(w):
    return unigram[w] / TOTAL

def P2(w1, w2):
    return bigram[(w1, w2)] / unigram[w1] if unigram[w1] else 0

def P3(w1, w2, w3):
    return trigram[(w1, w2, w3)] / bigram[(w1, w2)] if bigram[(w1, w2)] else 0

def predict(sentence, n, k=5):
    words = tokenize(sentence)
```

```

scores = []
for w in unigram:
    if n == 1:
        p = P1(w)
    elif n == 2 and words:
        p = P2(words[-1], w)
    elif n == 3 and len(words) >= 2:
        p = P3(words[-2], words[-1], w)
    else:
        p = 0
    scores.append((w, p))
return sorted(scores, key=lambda x: (-x[1], x[0]))[:k]

```

```

for n in (1, 2, 3):
    print("N =", n, predict("Artificial intelligence is", n))

```

```

print("Unseen bigram probability =", P2("is", "quantum"))
print("Unseen trigram probability =", P3("artificial", "intelligence", "quantum"))

```

Sample Demonstration and Evaluation

For “Artificial intelligence is”, the unigram model ranks words by overall frequency, while the bigram and trigram models use the preceding context. The exact top-5 list depends on the corpus. With the sample corpus, the model can predict words such as “changing”, “useful”, or “used” because those continuations occur in training.

Model	Context	Behavior
Unigram	None	Uses overall word frequency.
Bigram	Previous 1 word	Uses the immediately preceding word.
Trigram	Previous 2 words	Uses the preceding two words.

Prediction accuracy = correct predictions / total test cases × 100. Top-5 accuracy counts a case as correct when the actual next word occurs among the five predictions.

Limitations

- Unseen N-grams receive zero probability.
- Higher-order models are more sensitive to data sparsity.
- A small corpus cannot represent all valid English sequences.
- The model learns frequency patterns rather than meaning or world knowledge.
- Smoothing, backoff, or interpolation is needed for practical prediction.

Q2. Backoff and Interpolation for Language Prediction

Objective: Reduce the zero-probability problem by checking trigram, then bigram, then unigram estimates, and combine all three using Deleted Interpolation.

Python Implementation

Reuse unigram, bigram and trigram counters from Q1.

L1, L2, L3 = 0.2, 0.3, 0.5

```

def backoff_prob(w1, w2, w3):
    if trigram[(w1,w2,w3)]:
        return P3(w1,w2,w3)
    if bigram[(w2,w3)]:
        return P2(w2,w3)
    return P1(w3) if unigram[w3] else 0

```

```

def interpolation_prob(w1, w2, w3):
    return (L1 * P1(w3) +

```

$$L2 * P2(w2, w3) + \\ L3 * P3(w1, w2, w3))$$

```
def predict_backoff(sentence, k=5):
    words = tokenize(sentence)
    w1, w2 = words[-2], words[-1]
    scores = [(w, backoff_prob(w1, w2, w)) for w in unigram]
    return sorted(scores, key=lambda x: (-x[1], x[0]))[:k]
```

```
def predict_interpolation(sentence, k=5):
    words = tokenize(sentence)
    w1, w2 = words[-2], words[-1]
    scores = [(w, interpolation_prob(w1, w2, w)) for w in unigram]
    return sorted(scores, key=lambda x: (-x[1], x[0]))[:k]
```

```
print("Unsmoothed:", predict("Machine learning can", 3))
print("Backoff:", predict_backoff("Machine learning can"))
print("Interpolation:", predict_interpolation("Machine learning can"))
```

Comparison

Approach	Zero probabilities	Coverage	Main idea
Unsmoothed N-gram	High	Low on sparse data	Uses only the selected N-gram order.
Backoff	Reduced	Higher	Uses a lower-order model when the higher order is unavailable.
Interpolation	Reduced	High	Combines unigram, bigram, and trigram evidence.

Backoff is useful because an unseen trigram can still obtain a useful estimate from its bigram or unigram. Interpolation is useful because it combines general frequency information with contextual information, making the prediction less dependent on sparse high-order counts.

Q3. Entropy-Based Evaluation of Language Models

Objective: Measure predictive uncertainty using separate training and testing corpora and compare unigram, bigram, trigram, and smoothed models.

Concept

For a test sequence, cross-entropy can be estimated as $H = -(1/N) \sum \log_2 P(w_i | \text{context})$. Lower entropy means the model finds the sequence more predictable; higher entropy indicates greater uncertainty.

Python Implementation

```
import math
from collections import Counter
```

```
train = [
    "The cat sits on the mat.",
    "The cat eats food.",
    "The dog sits on the mat.",
    "The dog eats food.",
    "The cat likes the dog."
]
```

```
test = [
    "The cat sits on the mat.",
    "The quantum processor redesigned the system."
```



```
]
```

```
train_tokens = [tokenize(s) for s in train]
test_tokens = [tokenize(s) for s in test]
```

```
U, B, T = Counter(), Counter(), Counter()
for sent in train_tokens:
    U.update(sent)
    B.update(zip(sent, sent[1:]))
    T.update(zip(sent, sent[1:], sent[2:]))
```

```
V = len(U)
N = sum(U.values())
```

```
def smooth_u(w):
    return (U[w] + 1) / (N + V)
```

```
def smooth_b(w1,w2):
    return (B[(w1,w2)] + 1) / (U[w1] + V)
```

```
def smooth_t(w1,w2,w3):
    return (T[(w1,w2,w3)] + 1) / (B[(w1,w2)] + V)
```

```
def entropy(probs):
    return -sum(math.log2(max(p, 1e-12)) for p in probs) / len(probs)
```

```
def H1(words, smooth=False):
    probs = [smooth_u(w) if smooth else U[w]/N if U[w] else 1e-12 for w in words]
    return entropy(probs)
```

```
def H2(words, smooth=False):
    probs = []
    for i in range(1, len(words)):
        w1,w2 = words[i-1],words[i]
        p = smooth_b(w1,w2) if smooth else B[(w1,w2)]/U[w1] if B[(w1,w2)] else 1e-12
        probs.append(p)
    return entropy(probs)
```

```
def H3(words, smooth=False):
    probs = []
    for i in range(2, len(words)):
        w1,w2,w3 = words[i-2],words[i-1],words[i]
        p = smooth_t(w1,w2,w3) if smooth else T[(w1,w2,w3)]/B[(w1,w2)] if T[(w1,w2,w3)] else 1e-12
        probs.append(p)
    return entropy(probs)
```

```
for s in test_tokens:
    print("Sentence:", " ".join(s))
    print("Unigram:", H1(s))
    print("Bigram:", H2(s))
    print("Trigram:", H3(s))
    print("Smoothed unigram:", H1(s, True))
```

```
print("Smoothed bigram:", H2(s, True))
print("Smoothed trigram:", H3(s, True))
```

Interpretation

Test sentence	Relative uncertainty	Reason
The cat sits on the mat.	Low	The sequence is represented closely in the training corpus.
The quantum processor redesigned the system.	High	Important words and contexts are unseen in the small training corpus.

Exact entropy values depend on the training corpus and smoothing choice. Smoothing is important because it assigns small non-zero probabilities to unseen events, avoiding $\log(0)$ and allowing robust evaluation.

Q4. Comparative POS Tagging System

Objective: Assign grammatical tags using rule-based, stochastic, and transformation-based methods and demonstrate context-sensitive tagging.

Penn Treebank-Style Tags

Examples used in the design include NN (noun), VB (base-form verb), DT (determiner), PRP (personal pronoun), JJ (adjective), RB (adverb), VBD/VBG/VBZ (verb forms), and NNS (plural noun).

Python Implementation

```
import re
from collections import Counter, defaultdict

training = [
    [("I", "PRP"), ("book", "VB"), ("a", "DT"), ("ticket", "NN")],
    [("I", "PRP"), ("read", "VB"), ("a", "DT"), ("book", "NN")],
    [("She", "PRP"), ("reads", "VBZ"), ("the", "DT"), ("book", "NN")],
    [("The", "DT"), ("small", "JJ"), ("cat", "NN"), ("sits", "VBZ")],
    [("They", "PRP"), ("like", "VB"), ("good", "JJ"), ("books", "NNS")]
]

word_tag = defaultdict(Counter)
transition = defaultdict(Counter)
tag_count = Counter()

for sent in training:
    prev = "<START>"
    for word, tag in sent:
        word_tag[word.lower()][tag] += 1
        transition[prev][tag] += 1
        tag_count[tag] += 1
        prev = tag

lexicon = {w:c.most_common(1)[0][0] for w,c in word_tag.items()}

def rule_based(words):
    out = []
    for w in words:
        x = w.lower()
        if x in {"i", "he", "she", "they", "we", "you"}: tag = "PRP"
        elif x in {"a", "an", "the"}: tag = "DT"
        elif x.endswith("ly"): tag = "RB"
        elif x.endswith("ing"): tag = "VBG"
        elif x.endswith("ed"): tag = "VBD"
```

```

    elif x in lexicon: tag = lexicon[x]
    else: tag = "NN"
    out.append((w,tag))
return out

```

```

def stochastic(words):
    out, prev = [], "<START>"
    for w in words:
        best_tag, best_score = None, -1
        for tag in tag_count:
            emission = (word_tag[w.lower()][tag]+1)/(sum(word_tag[w.lower()].values())+len(tag_count))
            trans = (transition[prev][tag]+1)/(sum(transition[prev].values())+len(tag_count))
            score = emission * trans
            if score > best_score:
                best_tag, best_score = tag, score
        out.append((w,best_tag))
        prev = best_tag
    return out

```

```

def transformation_based(words):
    tags = [lexicon.get(w.lower(),"NN") for w in words]
    for i,w in enumerate(words):
        x = w.lower()
        if x == "i": tags[i] = "PRP"
        elif x in {"a","an","the"}: tags[i] = "DT"
        elif x == "book":
            if i == 1 and words[0].lower() == "i": tags[i] = "VB"
            elif i > 0 and words[i-1].lower() in {"a","an","the"}: tags[i] = "NN"
    return list(zip(words,tags))

```

```

def tag_sentence(sentence):
    words = re.findall(r"[A-Za-z]+", sentence)
    print("Rule:", rule_based(words))
    print("Stochastic:", stochastic(words))
    print("Transformation:", transformation_based(words))

```

```

tag_sentence("I book a ticket.")
tag_sentence("I read a book.")

```

Sample Demonstration

Sentence	Word	Contextual tag
I book a ticket.	book	VB – verb
I read a book.	book	NN – noun

The word “book” changes grammatical function according to context. In “I book a ticket”, it is a verb. In “I read a book”, it is a noun. This illustrates why POS tagging benefits from contextual information.

Evaluation

Tagging accuracy = correctly tagged tokens / total test tokens × 100.

```

def tagging_accuracy(predicted, gold):
    correct = sum(p == g for p,g in zip(predicted,gold))
    return 100 * correct / len(gold)

```

```

gold = ["PRP","VB","DT","NN"]
pred = ["PRP","VB","DT","NN"]
print("Accuracy:", tagging_accuracy(pred,gold), "%")

```

Comparison

Approach	Advantages	Limitations
Rule-based	Interpretable and easy to inspect.	Requires manually designed rules; can be brittle.
Stochastic	Learns word/tag and transition probabilities from data.	Needs representative tagged training data; sparse data can hurt performance.
Transformation-based	Applies interpretable corrective rules after initial tagging.	Depends on the quality of initial tags and transformation rules.

For a larger experiment, the compact demonstration corpus can be replaced with the Brown Corpus or another publicly available tagged English corpus, as permitted by the task statement.

Overall Conclusion

The four answers cover the required N-gram construction and prediction, backoff and interpolation, entropy-based language-model evaluation, and comparative POS tagging. Unsmoothed N-gram models are simple but suffer from data sparsity. Backoff and interpolation improve prediction coverage. Entropy measures predictive uncertainty. POS tagging shows that grammatical categories depend on both lexical evidence and context.

The numerical results will vary with the exact corpus and test cases used during execution; the supplied Python programs provide the required implementation framework and demonstrations.